

TABLE III
Decision Table

cbar $\geq$ x1	cbar $\leq$ x2	rbar $\geq$ y1	rbar $\leq$ y2	Binary	Decimal(=)	Move
0	0	0	0	0	0	X
0	0	0	1	01	1	X
0	0	1	0	10	2	X
0	0	1	1	11	3	X
0	1	0	0	100	4	X
0	1	0	1	101	5	Left
0	1	1	0	110	6	Left
0	1	1	1	111	7	Left
1	0	0	0	1000	8	X
1	0	0	1	1001	9	Right
1	0	1	0	1010	10	Right
1	0	1	1	1011	11	Right
1	1	0	0	1100	12	X
1	1	0	1	1101	13	Forward
1	1	1	0	1110	14	Backward
1	1	1	1	1111	15	Stop

```
'MarkerEdgeColor', 'b');
```

These statements pop-up a window where a 'blue' mark is plotted on the detected centre of mass of the red ball. They have been commented out in the main program for testing.

Plotting the position of the object

The position of the red ball is plotted as described below:

1. Divide the frame captured by the camera (refer Fig. 4) into five parts by means of points 'x1' and 'x2' on X-axis and points 'y1' and 'y2' on Y-axis.

2. Calculate x1, x2, y1 and y2 by the following method:

```
x1=x/2-numx; x2=x/2+numx  
y1=y/2-numy; y2=y/2+numy
```

'numx' and 'numy' are arbitrary numbers which you have to find out. These depend on the size of your ball. We have taken numx=120 and numy=30 in our program.

Calculate the coordinates of the centre of the frame, which is nothing but (x/2, y/2). 'x' is the maximum dimension of X-axis (in the program it is 640) and 'y' is the maximum dimension of Y-axis (in the program it is 480).

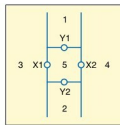


Fig. 4: Captured frame

3. Various conditions for detecting the position of the ball are:

- (i) If the ball is in region 5, it is at the centre of the frame. The robot should stop moving.
- (ii) If the ball is in region 3, it is at the left of the frame. The robot should move left.
- (iii) If the ball is in region 4, it is at the right of the frame. The robot should move right.
- (iv) If the ball is in region 1, it is at the upper part of the frame. The robot should move forward.
- (v) If the ball is in region 2, it is at the lower part of the frame. The robot should move backward.

Define the above conditions in the code for decision making as given below:

- (i) cbar $\geq$ x1 (output either '0' or '1')
- (ii) cbar $\leq$ x2 (output either '0' or '1')
- (iii) rbar $\geq$ y2 (output either '0' or '1')
- (iv) rbar $\leq$ y1 (output either '0' or '1')

4. If you consider these four conditions to be four bits, you can easily

convert the information into a number for easy computation. This can be done by the following code:

```
e=((cbar>= x1)*2*2*2 % bit number 3  
+(cbar<= x2)*2*2 % bit number 2  
+(rbar>= y1)*2 % bit number 1  
+(rbar<= y2) % bit number 0
```

Thereafter, you can simply generate a simple switch-case code for outputting the appropriate data through serial port, to control the external device such as a robot.

The decision table derived from the above conditions is shown in Table III. 'X' denotes a "don't care" situation. Some of the cases are imaginary in the practical world. The robot will stop in such situations.

For calculating the dimensions of the frame, use the sizeof() function. This function returns the size of the frame.

Setting the RGB threshold values

The red ball is of particular interest. The steps for setting the RGB threshold values of the red ball follow:

1. Take at least ten snaps of the red ball at various angles (refer Fig. 5)
2. Read each image
3. Display the image by using 'imview' function
4. Note down the pixel values of the red ball by placing the mouse cursor at various portions of the red ball. The threshold for red component (R_THRESHOLD) should be the least value of the red component found in the red ball. The threshold for green component (G_THRESHOLD) should be the maximum value of the green component found in the red ball. The threshold for blue component (B_THRESHOLD) should be the maximum value of the blue component found in the red ball.

In the screenshot shown in Fig. 5, the least value of the red component is '144.' So R_THRESHOLD can be taken as '144.' The maximum value of the green component is '115.' So G_THRESHOLD can be taken as '115.'